华东师范大学软件学院实验报告

实验课程：计算机网络	**年级：**_____2025 级_____	**实验成绩：**_____
实验名称：实验一 Protocol Layer（协议分层）	**姓名：**_____刘小凡_____	**实验日期：**_2026/9/15____
实验编号：1	**学号：**_____10244602411_____	**实验时间：14:50-16:25**

一、实验目的

本实验以一次 HTTP 网页获取为工作负载，通过 Wireshark 捕获并分析实际网络报文，理解协议分层、逐层封装与接收端分用的基本机制。重点掌握以下知识点：

- 理解 HTTP、TCP、IPv4 与 Ethernet II 在一次 Web 请求中的层次关系，以及“上层数据作为下层负载”的封装方式。
- 掌握 Wireshark 中报文列表、协议详情和十六进制字节区的基本使用方法，能够定位 HTTP GET 与 HTTP/1.1 200 OK 报文。
- 能够依据真实抓包数据确定 Ethernet、IP、TCP 首部长度，绘制 HTTP GET 报文的字节结构。
- 能够按实验手册定义估算下载方向的协议开销，并理解 Ethernet Type 与 IPv4 Protocol 字段作为分用键的作用。

二、实验内容与实验步骤

实验围绕“产生 HTTP 流量—捕获报文—检查协议层—分析报文结构—估算协议开销—验证分用键”展开。发送端的 HTTP 数据依次被 TCP、IPv4 和 Ethernet II 封装；接收端则根据各层首部中的类型字段逐层分用。

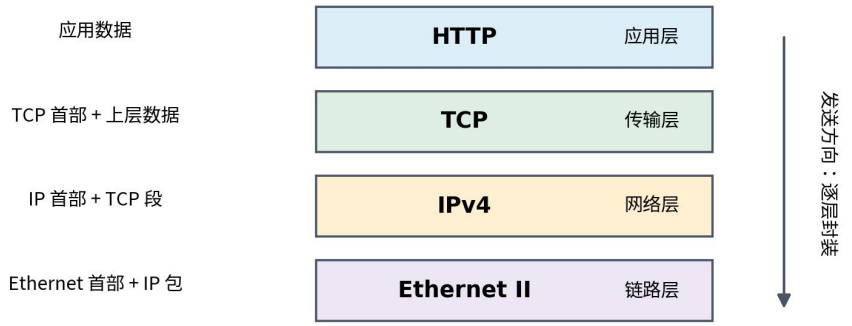


图 1 HTTP Web 获取过程中的协议分层与封装关系

步骤 1：在 WLAN 接口上启动 Wireshark 抓包，并使用 curl 访问 http://www.baidu.com 产生明文 HTTP/1.1 流量。

```
curl http://www.baidu.com
```

步骤 2：抓包结束后使用显示过滤器 http 定位 HTTP 报文，选择 Info 为 GET / HTTP/1.1 的请求报文，检查 Frame、Ethernet II、IPv4、TCP、HTTP 五个层次。

步骤 3：根据协议详情与原始字节，确定 GET 报文各层首部和应用数据的长度，绘制字节范围图。

步骤 4：分析服务器返回的 HTTP/1.1 200 OK，观察 TCP 分段与 Wireshark 重组，依据下载方向首轮有效传输估算 Ethernet/IPv4/TCP 首部开销。

步骤 5：检查 Ethernet II 的 Type 字段和 IPv4 的 Protocol 字段，回答分用键问题。

三、实验环境

项目	本次实验配置
操作系统与网络	Windows，WLAN 无线网络接口
抓包软件	Wireshark（本机已安装版本）
流量生成工具	curl 8.21.0
访问目标	http://www.baidu.com
客户端 IPv4	172.31.39.146
服务器 IPv4	182.61.200.110
应用/传输/网络/链路协议	HTTP/1.1 / TCP / IPv4 / Ethernet II

四、实验过程与分析

4.1 HTTP GET 报文定位与协议层次

在抓包文件中定位到客户端发往服务器的 HTTP GET 报文 No. 99227。该报文属于 tcp.stream = 95，从 172.31.39.146:57891 发往 182.61.200.110:80，Wireshark 依次解析出 Frame、Ethernet II、Internet Protocol Version 4、Transmission Control Protocol 和 Hypertext Transfer Protocol。Frame 是 Wireshark 对捕获记录的描述，并非网络协议；其余四项与 TCP/IP 分层结构一一对应。

为避免只展示单个报文，本报告先按 tcp.stream = 95 汇总 TCP 建连、HTTP GET、200 OK、重复段和连接关闭过程。

HTTP 会话关键报文流程（tcp.stream = 95）

基于 lab1.pcapng 中 Wireshark 解码结果；重复序列范围单独标注，不计入首次有效数据。

Frame / Direction	Event	TCP Payload	Frame Len	Analysis
97131 C→S	SYN	0 B	66 B	建立 TCP 连接
99206 S→C	SYN, ACK	0 B	66 B	确认连接请求
99227 C→S	**GET / HTTP/1.1**	**77 B**	**131 B**	**HTTP 请求；No. 99227**
101066-68 S→C	**HTTP/1.1 200 OK**	**789 + 1024 + 845 B**	**843 + 1078 + 899 B**	**首次有效响应，共 2658 B**
101119/102269-71 S→C	重复/重传段	845 / 789+1024+845 B	—	相同序列范围，不重复计入
102285 S→C	FIN, ACK	0 B	60 B	连接关闭

HTTP 响应通过三个首次有效 TCP 段完成重组；后续重复段说明真实网络中存在重传或重复到达。

图 2 HTTP 会话关键报文流程（tcp.stream = 95）

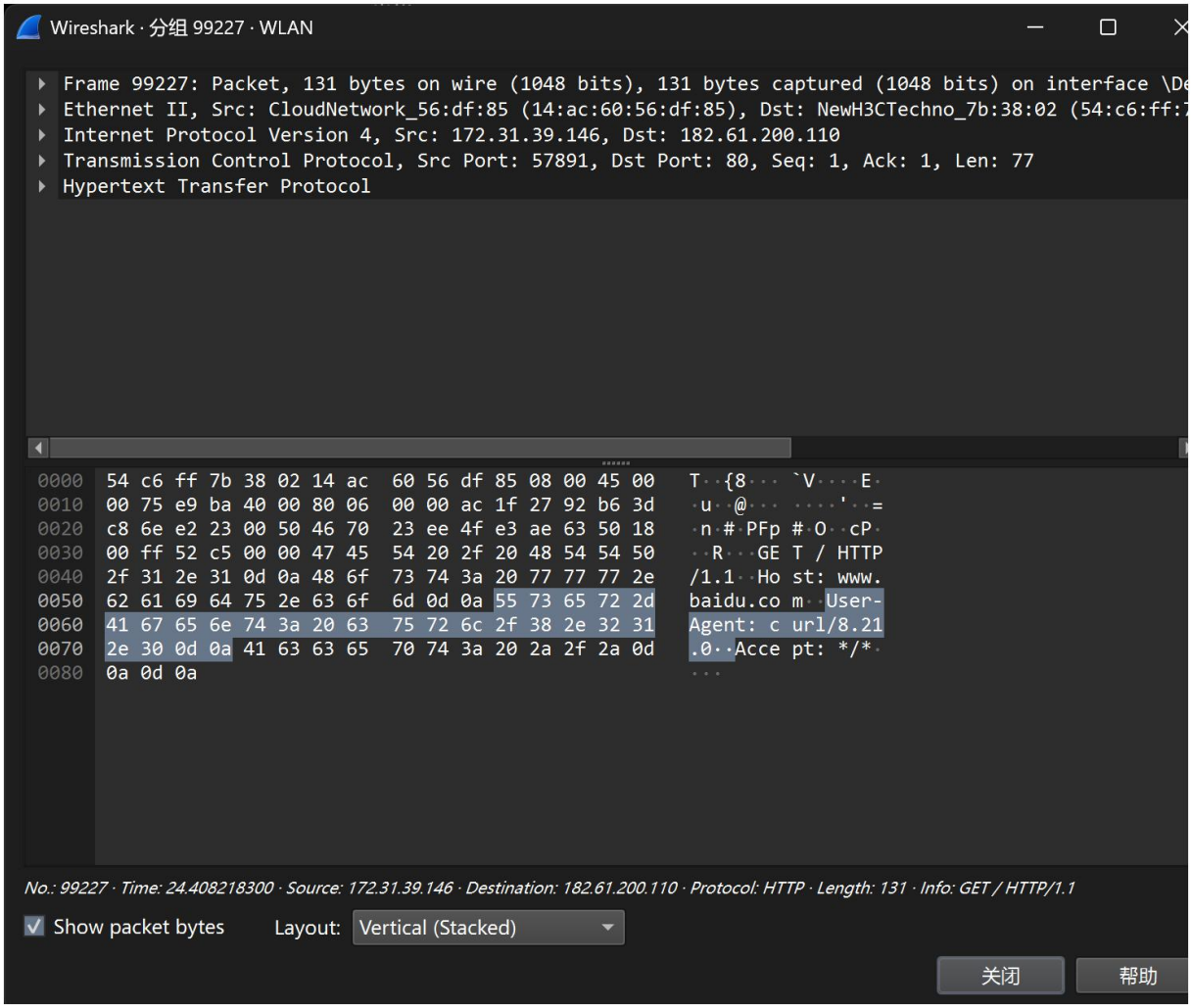


图 3 No. 99227 HTTP GET 报文的 Wireshark 协议详情与原始字节

4.2 GET 报文字段与长度分析

层次	关键字段	长度	分析
Frame	Packet 131 bytes on wire	131 B	整个捕获帧的长度。
Ethernet II	Src 14:ac:60:56:df:85；Dst 54:c6:ff:7b:38:02；Type 0x0800	14 B	0x0800 表明上层为 IPv4。
IPv4	Src 172.31.39.146；Dst 182.61.200.110；Total Length 117；Protocol 6	20 B	IHL=5，故首部长度为 5×4=20 B；Protocol=6 表示 TCP。
TCP	Src Port 57891；Dst Port 80；Payload Len 77	20 B	数据偏移为 5，故 TCP 首部为 20 B。
HTTP	GET / HTTP/1.1；Host: www.baidu.com；User-Agent: curl/8.21.0	77 B	应用层请求内容，作为 TCP payload。

由上表可得：131 B = 14 B（Ethernet）+ 20 B（IPv4）+ 20 B（TCP）+ 77 B（HTTP GET）。这一等式与 Wireshark 显示的帧长完全一致，可作为字段长度分析的交叉验证。

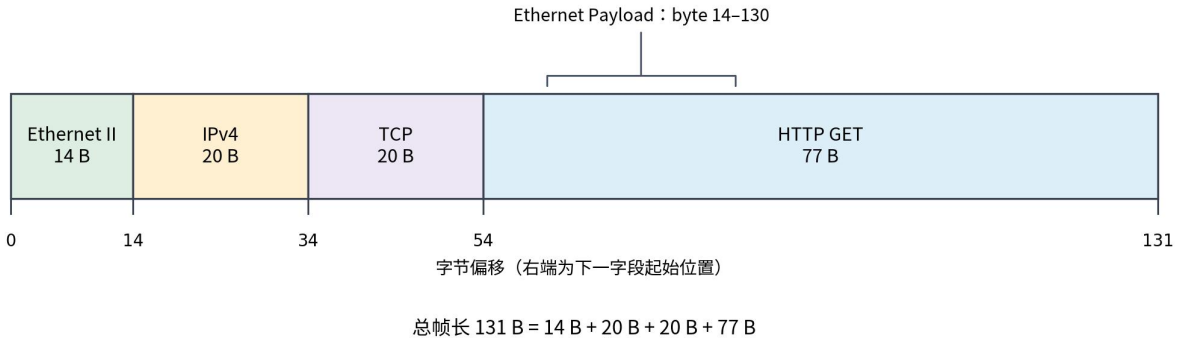


图 4 No. 99227 HTTP GET 报文的字节结构（基于本次真实抓包数据）

其中 Ethernet Header 占 byte 0-13，Ethernet Payload 为 byte 14-130；IPv4 Header 占 byte 14-33，IP Payload 为 byte 34-130；TCP Header 占 byte 34-53，HTTP GET 数据占 byte 54-130。左侧字段先在链路上传输，体现了低层首部位于高层数据之前的封装关系。

在 No. 99227 中展开 TCP 首部，可直接核对源端口 57891、目的端口 80、TCP Segment Len=77 和 Header Length=20 bytes。选中整个 TCP 协议项后，字节区高亮其 20 B 首部，与上表及 GET 字节结构图一致。

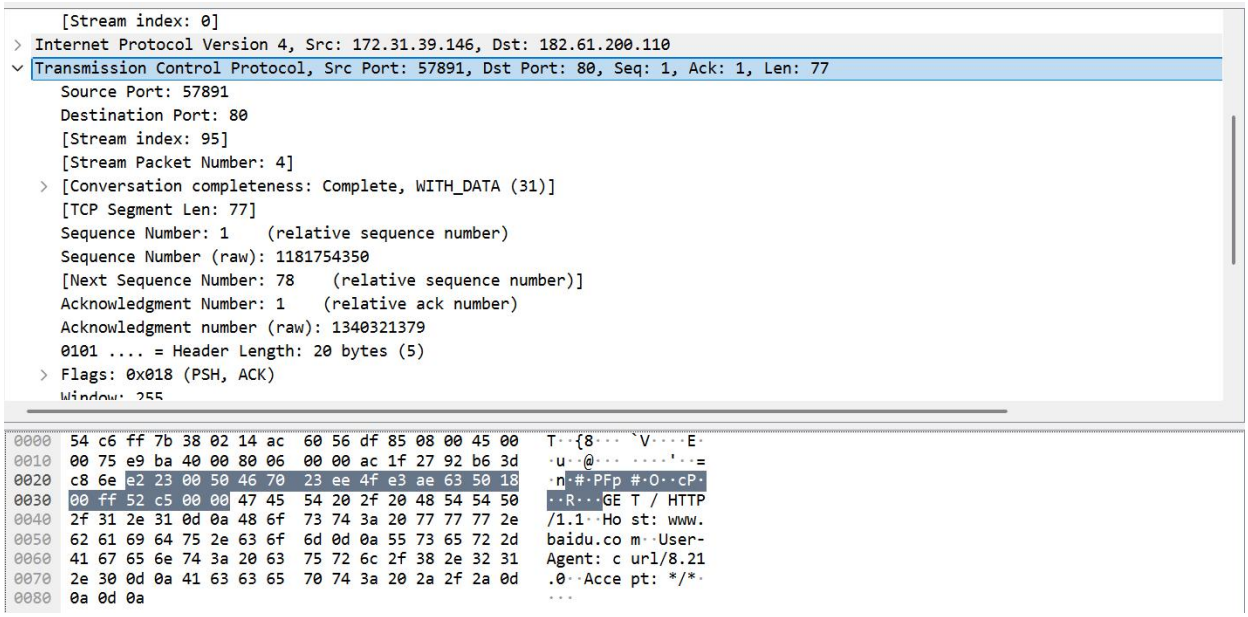


图 5 No. 99227 TCP 首部字段及 20 B 字节高亮

另选取下载响应 No. 101068 核对 IPv4 首部。其 Header Length=20 bytes，Total Length=885，Protocol=TCP (6)；加上 14 B Ethernet 首部，帧长为 899 B。该图展示的是响应报文，GET 请求的 IPv4 Total Length 仍为 117 B。

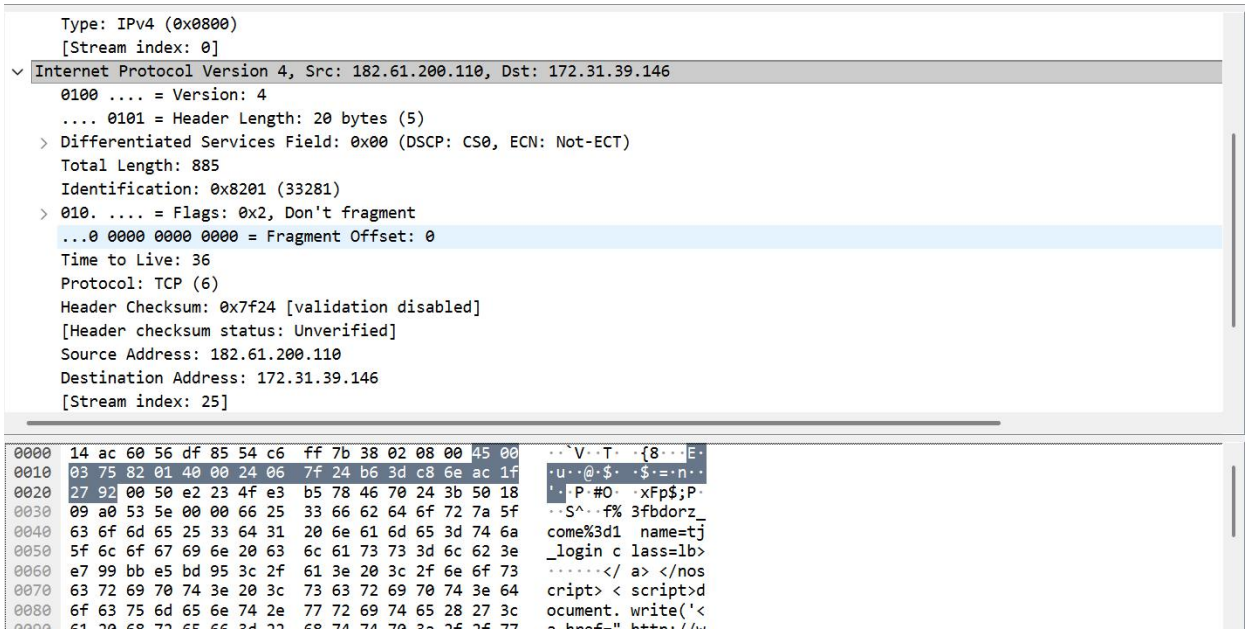


图 6 No. 101068 响应报文的 IPv4 首部及原始字节

4.3 HTTP 200 OK 与 TCP 分段重组

服务器返回 HTTP/1.1 200 OK。Wireshark 在最终可识别完整 HTTP 响应的位置显示“Reassembled TCP Segments”，说明一个 HTTP 消息被拆分到多个 TCP 段中传输，接收后再由 Wireshark 重组。 展开的重组详情显示 6 条分段记录，其中 No. 102270 与 No. 101067 的 payload 范围同为 789–1812，No. 101119、No. 102271 与 No. 101068 的范围同为 1813–2657。重复范围不增加唯一字节数，因此重组长度仍为 2658 B。

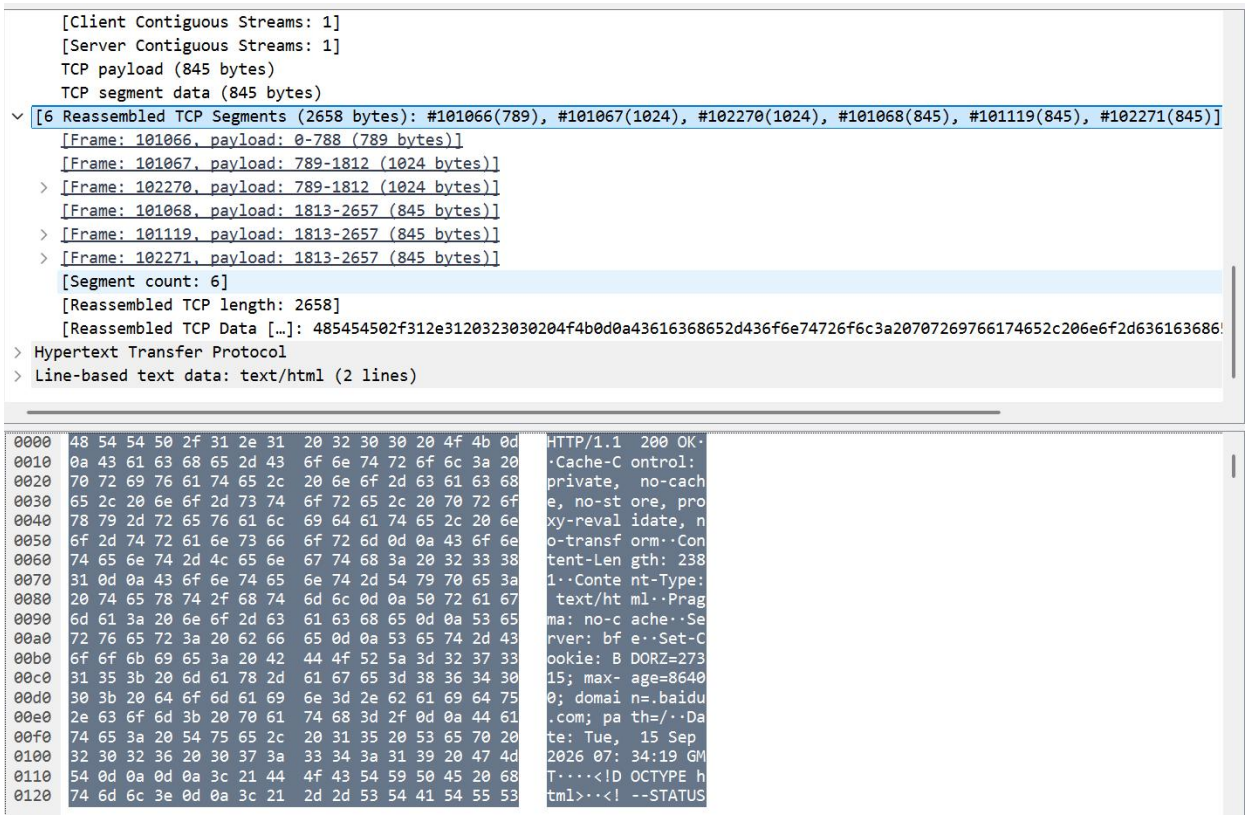


图 7 No. 101068 HTTP 响应重组及重复序列范围

报文编号	方向	TCP Payload	说明
101066	服务器 → 客户端	789 B	包含 HTTP/1.1 200 OK 响应头及正文起始部分
101067	服务器 → 客户端	1024 B	HTTP 正文中间部分
101068	服务器 → 客户端	845 B	HTTP 正文末尾部分

三个首次有效数据段的 TCP payload 之和为 789 + 1024 + 845 = 2658 B。由重组后的 HTTP 字节数减去 Content-Length = 2381 B，可得 HTTP 响应头长度为 277 B，因此 277 + 2381 = 2658 B，与 TCP 重组长度完全一致，说明重组结果正确。

4.4 下载方向协议开销估算

实验手册将 HTTP 头部和消息视为网络应承载的有效应用数据，将 Ethernet、IPv4、TCP 首部视为协议开销。为衡量协议本身带来的结构性开销，本报告按服务器→客户端方向的“首轮有效传输”统计，重复/重传的数据段不重复计入有效 HTTP 数据；重传现象在 4.6 中单独讨论。

报文	类型	Frame 长度	首部开销	HTTP 有效数据
99206	SYN, ACK	66 B	66 B	0 B
101065	ACK	60 B	54 B	0 B
101066	HTTP 数据	843 B	54 B	789 B
101067	HTTP 数据	1078 B	54 B	1024 B
101068	HTTP 数据	899 B	54 B	845 B

报文	类型	Frame 长度	首部开销	HTTP 有效数据
102284	ACK	60 B	54 B	0 B
102285	FIN, ACK	60 B	54 B	0 B
合计		**3066 B**	**390 B**	**2658 B**

注：60 B 的短 Ethernet 帧中存在最小帧长度带来的填充字节。实验手册将“TCP、IP、Ethernet headers”定义为 overhead，因此主结果只统计首部，不将 padding 并入首部开销。

本次首轮统计的协议首部开销为 390 B = 66 B + 6 × 54 B；有效 HTTP 数据为 2658 B。因此：

Protocol Overhead = 390 / (390 + 2658) × 100% ≈ 12.80%。

为提高数据可复核性，下面列出本次分析使用的显示过滤器、会话编号和关键报文长度。

使用 tcp.stream == 95 && tcp.srcport == 80 筛选下载方向，共显示 12 条捕获记录，其中包含重复数据段和 FIN 重传。下表所采用的 7 个首轮报文只是其中的子集，不能把 12 条记录全部当作首次有效传输。

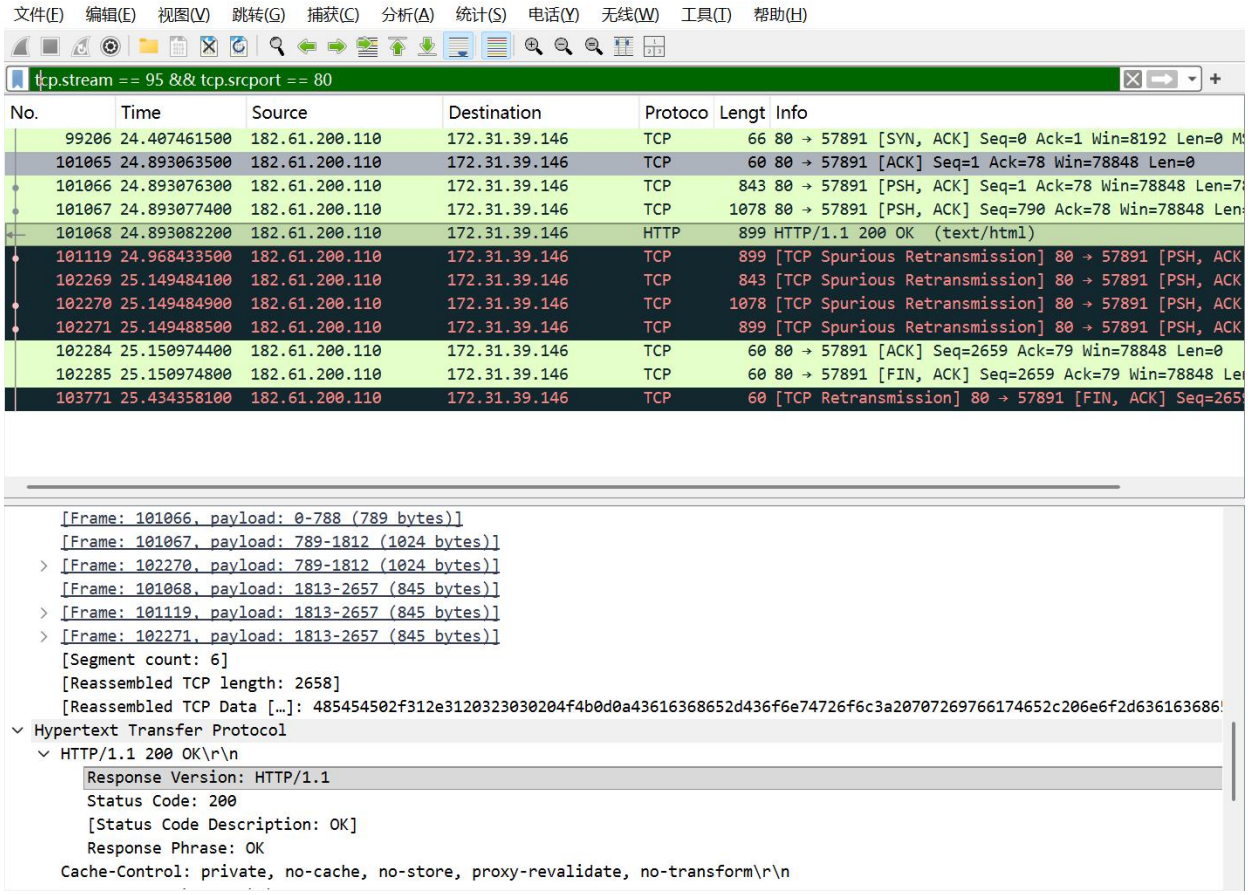


图 8 下载方向全部 12 条报文及 HTTP 200 OK



图 9　目标会话筛选与关键字段核对



图 10　下载方向有效数据与协议头开销占比

因此，按上述首次有效传输口径计算，协议首部开销约为 12.8%，HTTP 数据约占 87.2%；把所选短帧 padding 计入非应用数据时约为 13.3%。这两个比例对应表中所选的 7 个报文，不代表包含重传的全部 12 条下载记录。

4.5 Demultiplexing Keys（分用键）

层次	分用字段	本次取值	含义
Ethernet II	Type（EtherType）	0x0800	将 Ethernet payload 交给 IPv4 处理
IPv4	Protocol	6	将 IP payload 交给 TCP 处理

由此可见，协议层并不是依靠“猜测”来确定上层类型，而是通过首部中的显式类型字段完成逐层分用。这与发送端逐层封装正好构成对称过程。

No. 101068 的 IPv4 首部明确显示 Protocol: TCP (6)，验证 IPv4 将载荷交给 TCP 处理；前面的响应 IPv4 截图还显示 Ethernet Type: IPv4 (0x0800)，验证 Ethernet 到 IPv4 的分用。

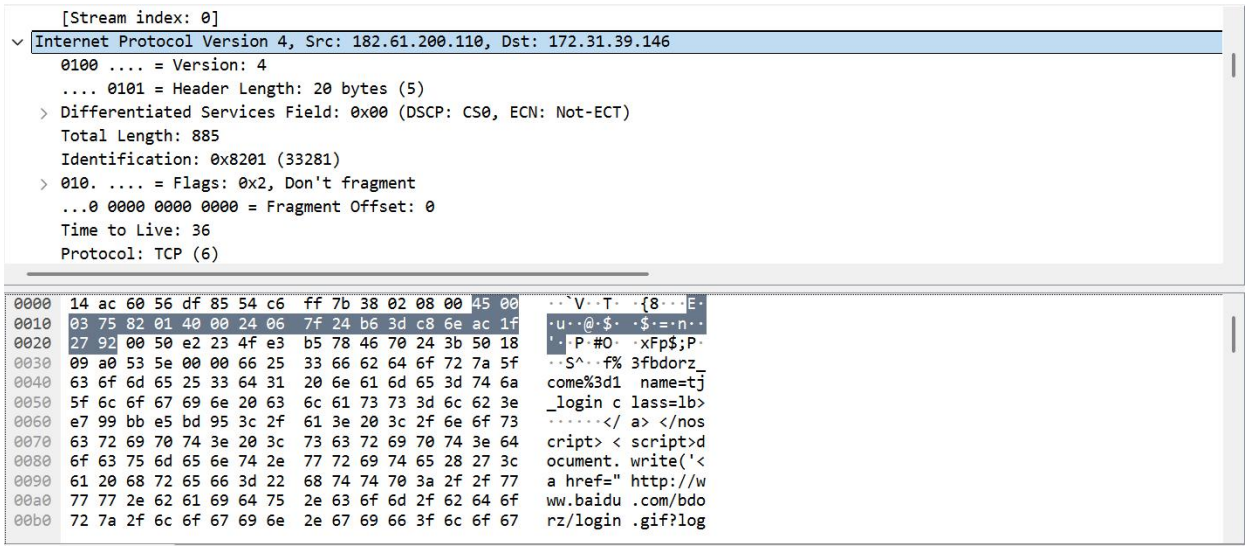


图 11 No. 101068 IPv4 Protocol 分用字段及首部高亮

将字段值画成逐层传递关系后，可以直观看出接收端如何依据首部字段把载荷交给下一层。

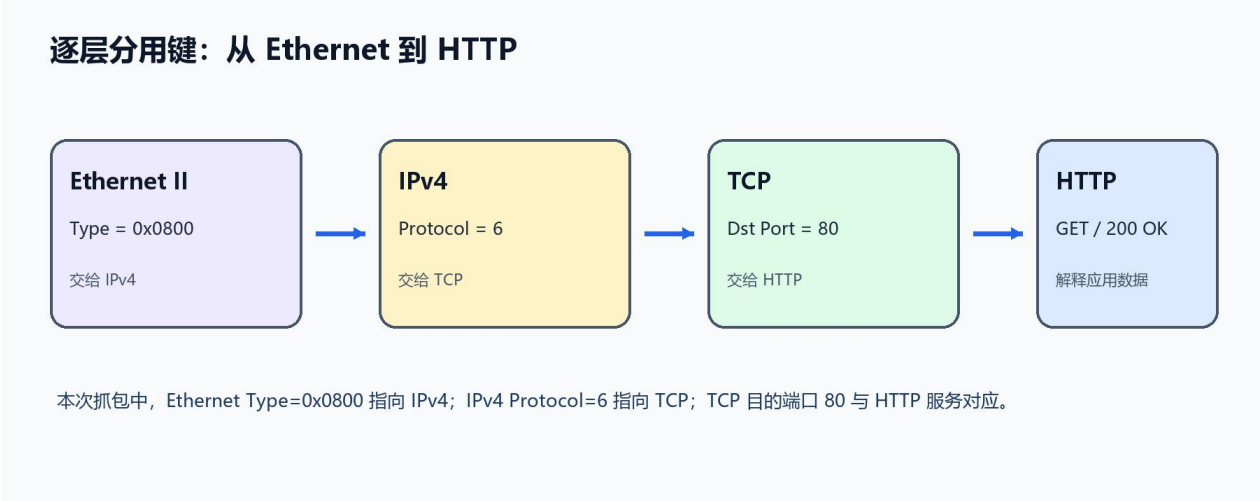


图 12 Ethernet、IPv4、TCP 到 HTTP 的逐层分用关系

4.6 实验现象、问题与分析

本次抓包中除首次有效的三个响应数据段外，还观察到相同 TCP Sequence Number 范围再次出现。例如 No. 101119 与 No. 101068 携带相同序列范围的数据；No. 102269–102271 又重复了此前的 789 B、1024 B、845 B 三段。该现象表明链路上存在重复段/重传，不能简单把所有 payload 相加作为“新的 HTTP 有效数据”。因此在协议开销估算中，本报告采用唯一序列范围对应的首次有效数据，并将重复段作为传输可靠性现象独立记录。

另外，实际抓包文件中存在较多背景流量。分析时通过 HTTP 显示过滤器以及端点 172.31.39.146:57891 ↔ 182.61.200.110:80 精确定位目标会话。后续实验可在开始抓包前进一步使用捕获过滤器 tcp port 80，以减少背景流量并提高分析效率。

若 Wireshark 对本机发出的 IP/TCP checksum 显示异常，也应优先考虑网卡 checksum offloading：操作系统在抓包点可能尚未由网卡填入最终校验和，因此不能据此直接判断报文损坏。

五、实验结果总结

（一）实验结果分析

本实验完成了从真实 Web 请求产生、Wireshark 抓包到逐层协议分析的完整过程。No. 99227 HTTP GET 报文总长为 131 B，其中 Ethernet II、IPv4、TCP 首部分别为 14 B、20 B、20 B，HTTP GET 数据为 77 B；该结果与原始字节和 Wireshark 协议详情相互验证，清楚体现了“高层数据逐层被低层首部封装”的协议分层机制。

服务器返回 HTTP/1.1 200 OK 后，Wireshark 显示响应由多个 TCP segment 承载，并在接收端完成重组。重组后的 HTTP 消息长度为 2658 B，其中 Content-Length=2381 B，说明除实体内容外还包含 HTTP 响应头等信息。这一现象说明应用层的一条完整消息并不要求与传输层的一条报文段一一对应，TCP 可以根据传输需要对字节流进行分段，并由接收端恢复为连续字节流。

按实验手册的定义，将 HTTP 头部与消息视为网络需要承载的有效数据，而将 Ethernet、IPv4、TCP 首部视为协议开销。本次下载方向首次有效传输中，协议首部开销估计约为 12.8%，即约 87.2% 的传输字节用于承载 HTTP 数据。由此可见，协议首部会产生一定额外开销，但在本次传输中并不占主导。Ethernet Type=0x0800 与 IPv4 Protocol=6 还分别验证了 Ethernet→IPv4、IPv4→TCP 的分用过程。

实验过程中还观察到重复 TCP 序列范围及背景网络流量，说明真实网络抓包比教材中的理想化报文序列更复杂。分析时必须结合源/目的 IP、端口、Sequence Number 以及 Wireshark 的重组提示来区分有效数据、控制报文和重复段，不能简单把所有捕获到的 payload 直接相加。

（二）（二）Turn-in 题回答

1. Ethernet 层的 demultiplexing key 是 Type（EtherType）字段；本次报文中 0x0800 表示上层协议为 IPv4。

2. IPv4 层的 demultiplexing key 是 Protocol 字段；本次报文中数值 6 表示上层协议为 TCP。

3. HTTP GET 报文结构：Ethernet 14 B + IPv4 20 B + TCP 20 B + HTTP 77 B = 131 B。

4. 下载方向协议首部开销估计：约 12.8%。结论：协议首部会带来一定开销，但本次传输中开销不显著，大多数传输字节用于承载 HTTP 数据。

（三）思考题回答（Explore on your own）

思考题 1：一个不携带高层数据的短 TCP 报文究竟发给谁？既然没有 HTTP 等高层数据，它有什么作用？

这类报文仍然有明确的接收对象：它由 IP 层送达对端主机后，再依据 TCP 源/目的端口交给对端的 TCP 实体（具体 TCP 连接）处理，而不是交给 HTTP 去解释应用数据。即使 TCP payload 长度为 0，TCP 首部中的 ACK、SYN、FIN、RST、Window 等字段仍可完成连接建立与释放、确认已收到的数据、流量控制和异常复位等功能。例如纯 ACK 报文没有应用层数据，却是 TCP 可靠传输机制的重要组成部分。

思考题 2：同一个 HTTP 响应被拆成多个 TCP 报文段时，承载响应的第一个 TCP 报文和最后一个 TCP 报文在结构图上会有什么不同？

两者的基本封装层次仍相同，通常都表现为 Ethernet Header + IP Header + TCP Header + TCP Payload；差别主要在 TCP payload 的内容与长度、Sequence Number 等传输层字段。第一个报文段承载 HTTP 响应的起始部分，通常包括 HTTP 响应头和正文开头；中间报文段继续承载后续字节；最后一个报文段承载消息尾部，其 payload 往往可能短于中间的满长度报文段。由于 TCP 面向字节流，Wireshark 只有在收齐足够的分段后才能还原完整 HTTP 消息，因此前面的分段可能只显示为 TCP，而在完成重组的报文处会出现 HTTP 以及“Reassembled TCP Segments”的提示。本次抓包中的 200 OK 报文正观察到了这种重组现象。

思考题 3：如果某一较低层除了添加首部，还对来自高层的数据进行加密，经典“逐层加首部”的模型应如何修改？

此时低层不再只是把自己的首部简单拼接到高层消息前面，而会先（或在封装过程中）把高层数据转换为密文，再携带必要的加密相关信息，例如随机数/Nonce、初始化向量、认证标签或填充信息。在线路上看到的该层 payload 将是经过加密后的字节，而不是原始高层数据。接收端则先由这一层完成完整性验证和解密，再把恢复出的高层消息交给上一层。因此，分层思想仍成立，但“封装”需要理解为“添加控制信息 + 对 payload 进行变换”，而不只是机械地添加首部。

思考题 4：如果某一较低层会对来自高层的数据进行压缩，经典分层模型又应如何变化？

压缩与加密类似，也属于对高层 payload 的变换。低层接收到高层消息后，先将其编码为压缩后的表示，再添加本层所需的首部或压缩参数信息；接收端执行相反操作，先解压缩，再将恢复后的高层数据交给上一层。这样一来，低层消息的长度与高层消息长度不再是“高层长度 + 固定首部长度”的简单关系：压缩效果好时总长度会减小，若原始数据很短或不可压缩，也可能因为压缩元数据而没有明显收益。

（四）实验心得体会

通过本次实验，最直观的收获是把教材中的 HTTP、TCP、IP、Ethernet 分层从抽象框图落实到了真实报文字节上。点击同一个 HTTP GET 报文即可依次看到 Ethernet、IPv4、TCP 和 HTTP，并能通过字节偏移验证每一层首部的实际长度；这使“封装、解封装、分用”等概念不再只是定义，而成为可以直接观察和验证的网络行为。

同时，实验也体现了真实网络分析需要比理论模型更加谨慎：HTTP 响应会跨多个 TCP segment，抓包中可能出现重传、纯 ACK、背景流量以及校验和卸载等现象。后续实验中应在抓包前设置更精确的 capture filter，并结合五元组、序列号和 Wireshark 的重组信息进行分析；还可选择不同大小的 HTTP 资源重复实验，比较应用数据规模变化时协议固定开销所占比例的变化。通过这次实践，我初步掌握了使用 Wireshark 从“现象—字段—协议机制”三个层面分析网络通信的方法。

六、附录

附录 A：关键命令与显示过滤器

```
curl http://www.baidu.com
```

```
http
```

ip.addr == 172.31.39.146 && ip.addr == 182.61.200.110

```
tcp.stream == 95
```

tcp.stream == 95 && tcp.srcport == 80

frame.number == 101068

附录 B：数据来源说明

本报告的报文编号、IP 地址、端口、首部长度、TCP payload 长度、HTTP Content-Length 及重传现象均来自本次实验实际生成的 Wireshark 抓包文件 lab1.pcapng；过滤和定位过程使用显示过滤器 http 以及目标端点 172.31.39.146:57891 ↔ 182.61.200.110:80。